

SecureBank Mobile

A Self-Directed Agile Simulation Project

Scrum Master Portfolio Case Study

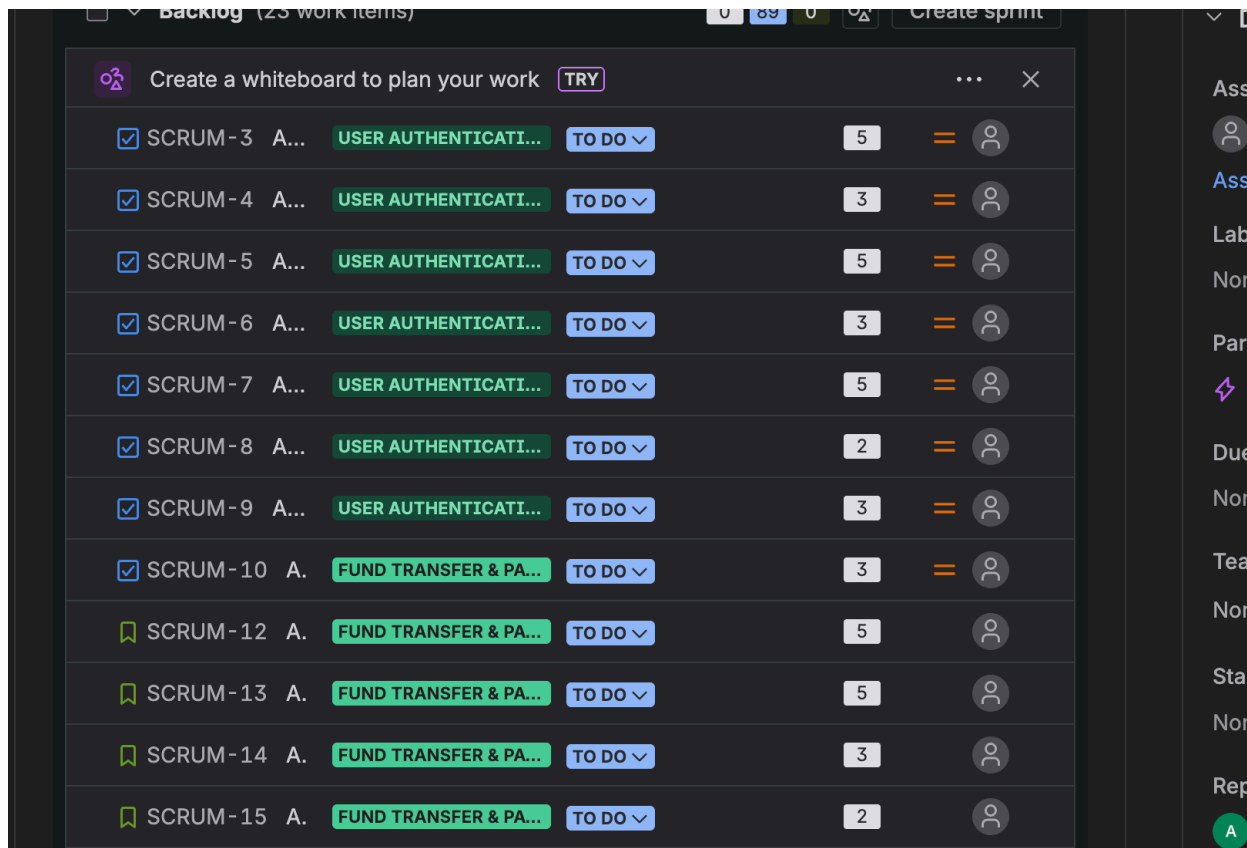
Project Overview

SecureBank Mobile is a self-directed simulation project built to demonstrate hands-on Scrum Master skills using Jira, Confluence, and Python. The project simulates a mobile banking application release across two completed sprints, covering the full Scrum lifecycle: backlog creation, sprint planning, active sprint facilitation, mid-sprint scope management, capacity recalibration using real velocity data, sprint review, retrospective, and custom reporting.

This project was built to bridge a background in mainframe QA testing with Certified Scrum Master (CSM) training, providing concrete, demonstrable artifacts rather than theoretical framework knowledge.

Product Backlog

The product backlog was structured into 4 epics covering the core functionality of a banking app: User Authentication & Security, Fund Transfer & Payments, Fraud Detection Alerts, and Account Dashboard & Statements. Each of the 24 user stories includes story point estimates (Fibonacci scale) and Given/When/Then acceptance criteria.



Full product backlog in Jira — 4 epics, 24 stories, story points assigned

Sprint 1 Planning

Sprint 1 focused on delivering core authentication functionality first, reflecting a dependency-first prioritization approach — security features were treated as a prerequisite for payment and fraud features built in later sprints.

Metric	Value
Sprint Length	2 weeks (10 working days)
Team Composition	5 Developers, 2 QA, 1 Product Owner
Estimated Capacity	25–30 story points (first sprint, no prior velocity)
Committed Points	26 story points (7 stories)

The screenshot shows a Jira workspace for 'SecureBank mobile'. The main panel displays the 'SCRUM Sprint 1' backlog with 7 work items, each with a 'TO DO' status and an estimate. The right sidebar shows the 'Jira work item' details for 'SCRUM-2 User Authentication & Security'.

Item	Summary	Status	Estimate
SCRUM-3	As a user, I want to log in using f...	TO DO	5
SCRUM-4	As a user, I want to log in using f...	TO DO	3
SCRUM-5	As a user, I want to set up multi-...	TO DO	5
SCRUM-6	As a user, I want to receive an al...	TO DO	3
SCRUM-7	As a user, I want to reset my pas...	TO DO	5
SCRUM-8	As a user, I want my session to a...	TO DO	2
SCRUM-9	As a compliance officer, I want a...	TO DO	3

7 of 7 work items visible | Estimate: 26 of 26

Backlog (16 work items) | Estimate: 63 of 0

SCRUM-10 As a ... FUND TRANSFER & PA... TO DO 3

SCRUM-12 As a ... FUND TRANSFER & PA... TO DO 5

Details for SCRUM-2 User Authentication & Security:

- Assignee: Unassigned
- Labels: None
- Parent: SCRUM-2 User Authentication & Security
- Due date: None

Sprint 1 backlog — 7 stories from the Authentication epic, 26 committed points

Mid-Sprint Scope Management

Partway through Sprint 1, a critical fraud-detection requirement was escalated by the Product Owner: flagging logins from unrecognized countries. As Scrum Master, I facilitated a scope discussion to protect the sprint goal without exceeding team capacity.

Resolution: SCRUM-6 (New Device Alert, 3 points — overlapping scope, lower urgency) was moved back to the backlog to make room for the new requirement, keeping the sprint's committed points unchanged at 26.

SecureBank mobile

Summary

Backlog

Board

Calendar

Timeline

More 2

Search backlog

A

Filter

SCRUM-3

As a user, I want to log in using f...

IN PROGRESS

5

SCRUM-4

As a user, I want to log in using f...

IN PROGRESS

3

SCRUM-5

As a user, I want to set up multi-...

TO DO

5

SCRUM-6

As a user, I want to receive an al...

TO DO

3

SCRUM-7

As a user, I want to reset my pas...

TO DO

5

SCRUM-8

As a user, I want my session to a...

DONE

2

SCRUM-9

As a compliance officer, I want a...

IN PROGRESS

3

SCRUM-30

"Critical: Add fraud alert fo...

TO DO

3

+ Create

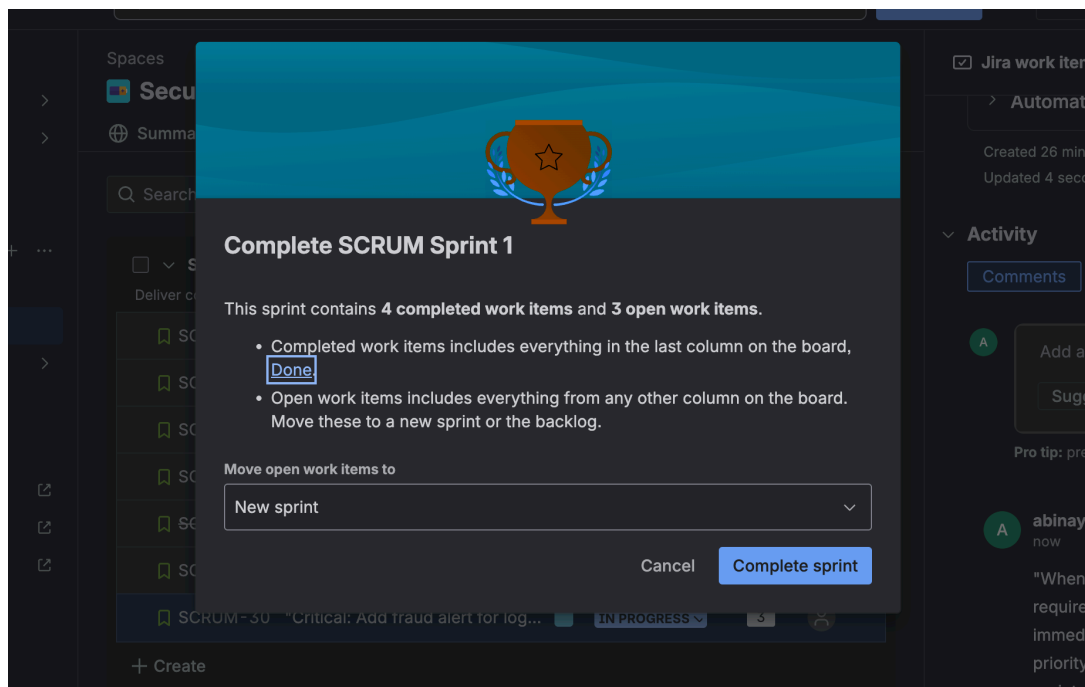
Backlog (16 work items)

0630

Create sprint

Sprint board mid-scope-change — new critical story added, lower-priority story swapped out

Sprint 1 Completion & Results



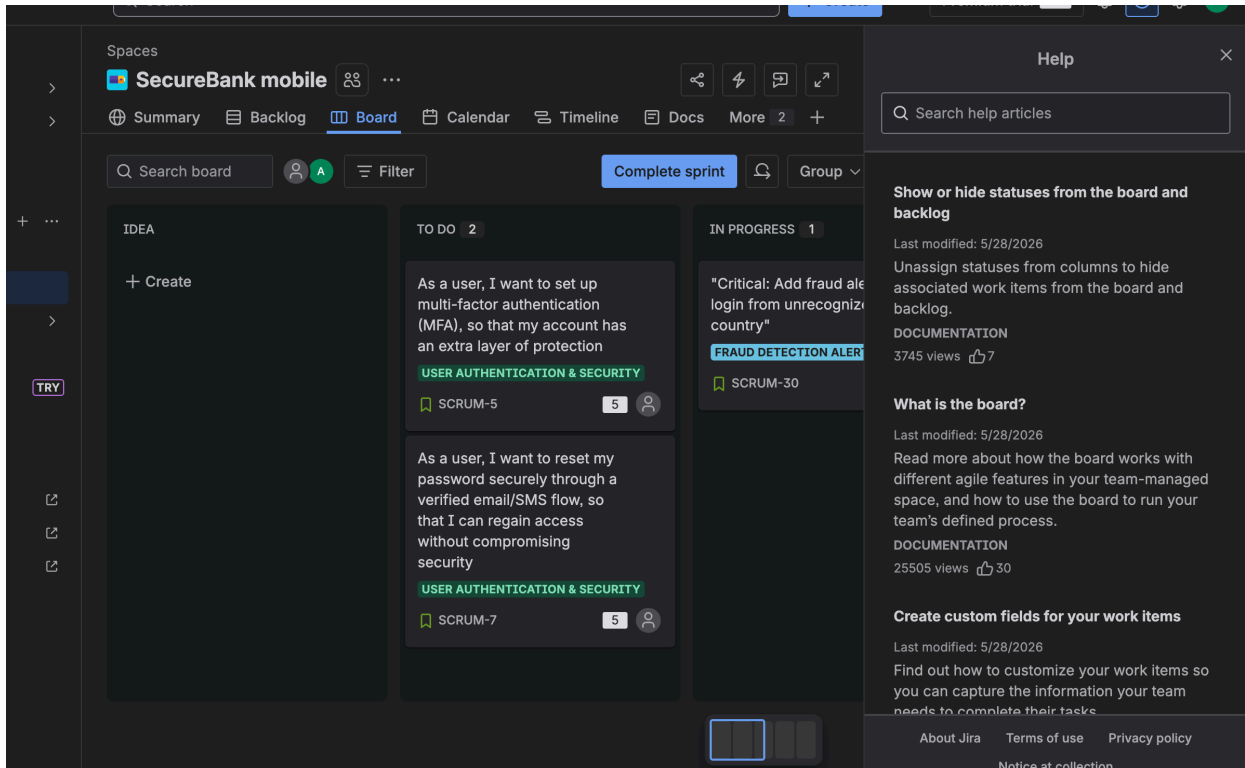
Sprint 1 completion summary — 4 completed items, 3 carried forward to Sprint 2

Metric	Result
Committed	26 points (7 stories)
Completed	13 points (4 stories)
Carried Over	13 points (3 stories) → Sprint 2
Completion Rate	50%

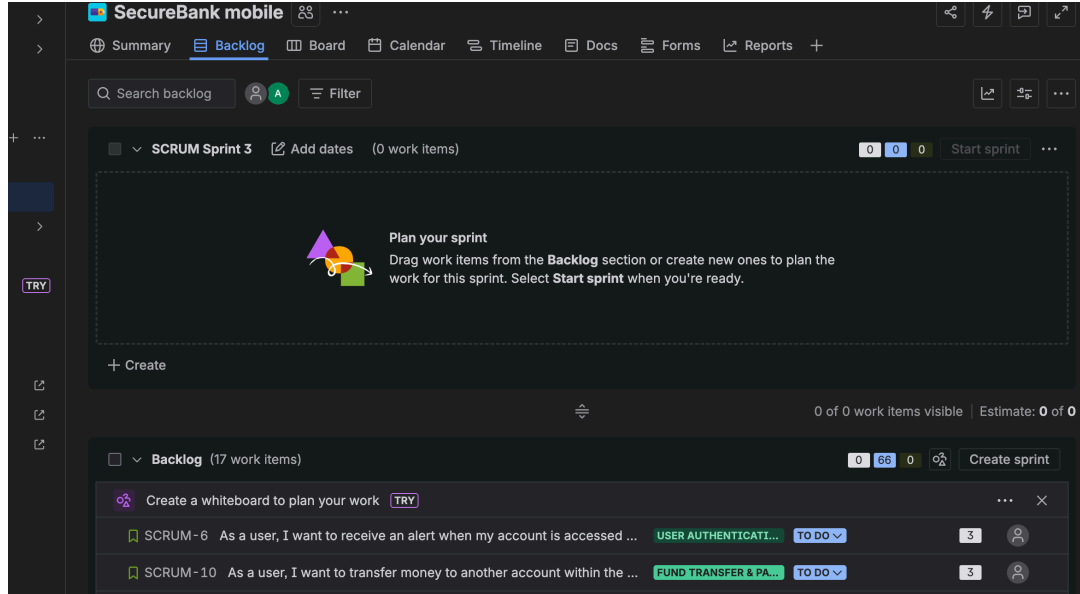
Sprint 2: Recalibration and Delivery

Sprint 2 focused exclusively on completing the three stories carried over from Sprint 1. Rather than layering in new scope, capacity was recalibrated using Sprint 1's actual velocity (13 points) instead of the original untested estimate of 25–30 points.

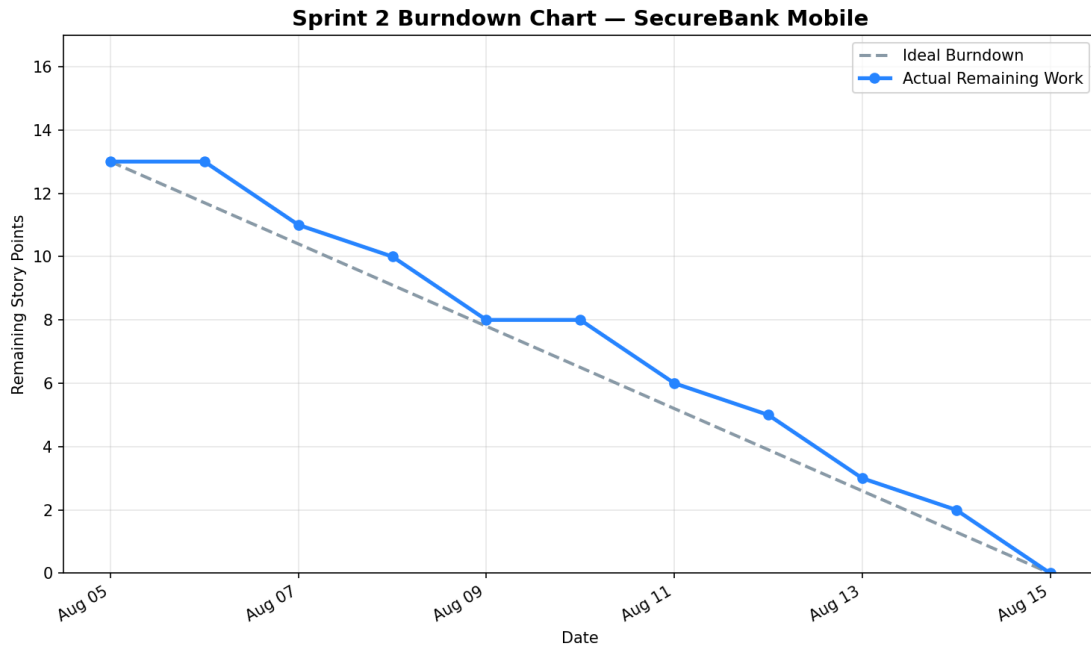
Metric	Result
Capacity Basis	Sprint 1 actual velocity (13 points)
Committed	13 points (3 stories, all carried over)
Completed	13 points (3 stories)
Completion Rate	100%



Sprint 2 board — carried-over stories from Sprint 1 in progress



Sprint 2 completed with 0 open items — full carryover scope delivered

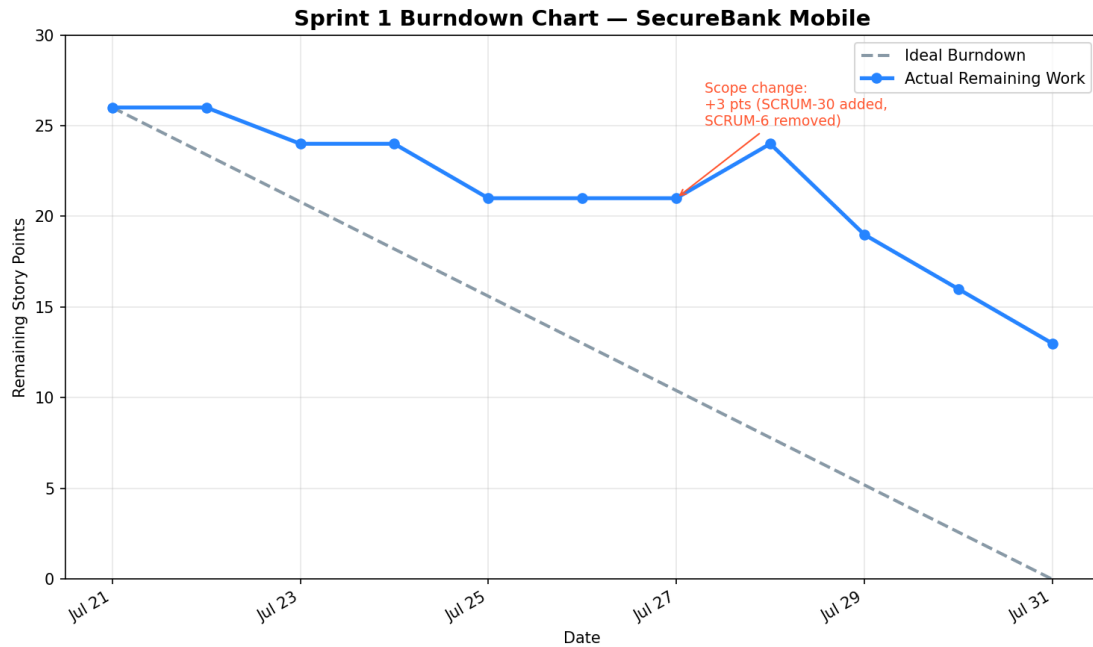


Sprint 2 burndown — a clean, steady completion with no scope disruption

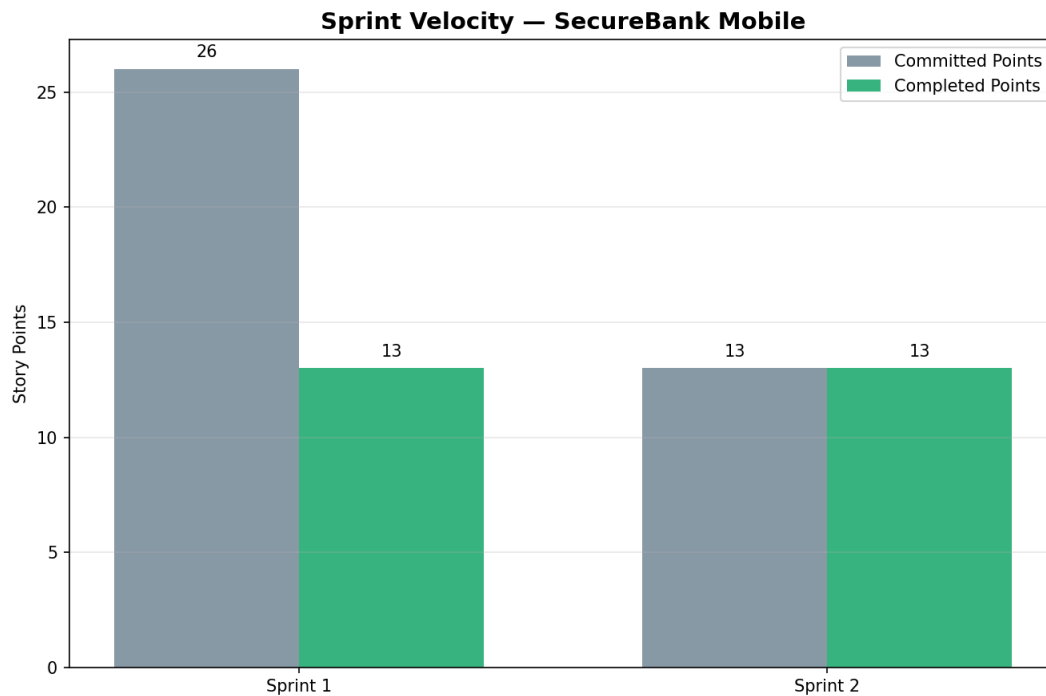
This result validated the recalibration decision: when sprint capacity is grounded in real velocity data rather than a first-sprint guess, the team can commit to and deliver a fully achievable scope.

Custom Sprint Reporting (Python)

The team's Jira plan did not include native burndown or velocity reporting, so I built a lightweight Python reporting tool (matplotlib) to generate both charts directly from sprint data. The script is reusable — future sprints are added by updating a data dictionary and re-running the script.



Sprint 1 burndown chart — ideal vs. actual, with the mid-sprint scope change annotated



Sprint velocity — Sprint 1 (26 committed / 13 completed) vs. Sprint 2 (13 committed / 13 completed)

Sprint 1 Retrospective Highlights

What went well:

- Core biometric authentication features (fingerprint, Face ID) delivered on schedule
- Team responded quickly to a mid-sprint critical security requirement without derailing the sprint goal
- Compliance-sensitive audit logging feature completed and verified

What didn't go well:

- Completion rate (50%) came in below the initial capacity estimate, largely due to the mid-sprint scope change
- Two originally planned stories (MFA setup, password reset) were not started

Action items carried into Sprint 2:

- Prioritize the 3 carried-over stories at the start of Sprint 2
- Reserve ~10% capacity buffer in future sprint planning to absorb urgent requirements
- Recalibrate Sprint 2 capacity using Sprint 1's actual velocity (13 points) instead of the initial estimate

Sprint 2 Retrospective Highlights

What went well:

- 100% sprint completion — all 3 carried-over stories delivered
- Capacity recalibration based on real velocity data proved accurate
- No new work-in-progress was started until carryover items were closed out, avoiding scope creep

What didn't go well:

- Sprint scope was intentionally conservative; no new epics were advanced this sprint

Action items carried into Sprint 3:

- Begin pulling stories from the Fund Transfer & Payments epic
- Use a 13–15 point capacity baseline, informed by two consecutive sprints of data

Skills Demonstrated

- Sprint planning, backlog grooming, and story point estimation
- Facilitating mid-sprint scope trade-off decisions
- Recalibrating team capacity using real velocity data across sprints
- Sprint review and retrospective documentation
- Jira administration: epics, stories, sprints, and board management
- Confluence documentation for planning and review artifacts
- Python scripting for custom sprint metrics and reporting

Full backlog, planning docs, and reporting script available on request.